

●●●

Seed: Vanilla RAG

80 lines

```

class KnowledgeBase:
    def write(self, item):
        chunks = self._chunk(text)
        self.collection.add(
            documents=chunks)
    def read(self, query):
        return collection.query(
            query_texts=[q])

```

📄

Single-field storage

🔍

Similarity retrieval

#

No scoring or ranking

📄

x3 seed variants

	Evaluated on		
Program ↓	LoCoMo	ALFWorld	PRBench
Seed	0.256	0.720	0.466
Dial. Contacts	0.408	0.620	0.055
Task Playbook	0.021	0.880	0.576
Legal Database	0.086	0.580	0.660

Table 3: Cross-task transfer. Bold = native task.

●●●

LoCoMo: Dialogue Contacts

273 ln

```

# Write: extract per-person facts
entities = item.entities
facts = item.key_facts # 3-8 atomic
db.execute("INSERT INTO kb...")

# Read: entity-name scoring dominates
for row in all_rows:
    exact = sum(1 for e in q_ent
        if e.lower() in row.entities)
    soft = sum(1 for e in q_ent
        if e.lower() in row.text)
    score = exact*18 + soft*8 + overlap*2
    if answer_focus & row.terms:
        score += 6 # topic bonus

```

📄

Per-person fact extract

🔍

Entity scoring (x18)

📄

Query-focused selection

⚡

Lexical only, no LLM

●●●

ALFWorld: Task Playbook

237 ln

```

# Schema: task decomposition slots
class KnowledgeItem:
    task_type: str # "pick_clean_place"
    target_object: str # "soapbottle"
    required_state: str # "clean"
    target_receptacle: str
    pitfalls: str # common mistakes
    aliases: list # ["soap", "bottle"]

# Read: slot matching, no LLM
if q_task in doc_task:
    score += 2.0
if q_state in blob:
    score += 1.5

```

📄

Task-slot schema

🔍

Exact slot matching

🛡️

Pitfall & alias track

⚡

Fully deterministic

●●●

PRBench: Legal Database

679 ln

```

# Write: auto-extract legal citations
clues = _extract_clues(raw_text)
auth = [c for c in clues
    if "/" in c or c.startswith("Art")]
db.execute("INSERT...", (topic,
    jurisdiction, *11_fields, raw))

# Read: jurisdiction as primary sort
juris = jurisdiction_score(
    q.hint, rec["jurisdiction"])
score = overlap + 2.0*detail + 1.8*entity
score += juris
ranked.sort(key=lambda x: x.score)
return llm_synthesis(top_8)

```

📄

14-col table + indexes

📄

Citation extraction

🏆

Jurisdiction-first rank

⚡

LLM synthesis